Visualização de Dados Avançada com Seaborn

Asimov Academy

ASIMOV

Conteúdo

01. Introdução ao Seaborn **3**
Principais Características . . . 3
Documentação e Recursos Adicionais . . . 3

02. Plots de distribuição **4**
Importando a Biblioteca Seaborn . . . 4
Carregando Conjunto de Dados . . . 4
Gráficos de Distribuição . . . 5
Histograma com `sns.histplot` . . . 5
Gráfico de Distribuição Conjunta com `sns.jointplot` . . . 6
Gráfico de Correlação com `sns.pairplot` . . . 8
Rug Plot com `sns.rugplot` . . . 10
Conclusão . . . 11

03. Plots categóricos **12**
Gráficos de Barras: `barplot` e `countplot` . . . 12
Boxplot . . . 13
Violinplot . . . 15
Swarmplot . . . 16
Conclusão . . . 18

04. Plots de regressão **19**
O que é o LMPlot? . . . 19
Personalização com o parâmetro hue . . . 20
Ajustando Marcadores . . . 21
Modificando o Tamanho dos Pontos . . . 22
Separando os Gráficos em Colunas . . . 23
Segmentação por Linhas . . . 24
Ajustando Dimensões do Gráfico . . . 24
Conclusão . . . 25

05. Plots Matriciais **26**
Transformação dos Dados . . . 26
Criando um Heatmap . . . 26
Ajustando o Tamanho do Gráfico . . . 27
Personalização do Heatmap . . . 27
Cluster Map . . . 28

Conclusão 29

06. Estilização **30**

Função `set_style` 30

Estilo `ticks` 31

Estilo `darkgrid` 32

Estilo `dark` 33

Estilo `whitegrid` 34

Customização com Matplotlib 35

Configuração de Contexto com `set_context` 35

Considerações Finais 36

01. Introdução ao Seaborn

O Seaborn é uma biblioteca de visualização de dados em Python que simplifica a criação de gráficos estatísticos de alta qualidade. Ele é amplamente utilizado por cientistas de dados, analistas e qualquer pessoa que precise explorar e comunicar padrões em conjuntos de dados. A biblioteca é construída sobre o Matplotlib e oferece uma interface mais intuitiva para criar visualizações informativas com menos linhas de código.

Principais Características

- **Facilidade de uso**: Permite criar visualizações complexas com poucos comandos.
- **Foco estatístico**: Inclui ferramentas para visualizar distribuições, correlações e relações entre variáveis.
- **Integração com Matplotlib**: Usa essa biblioteca como base, permitindo ajustes adicionais.
- **Estilo aprimorado**: Produz gráficos visualmente agradáveis por padrão, com opções avançadas de customização.

Documentação e Recursos Adicionais

Para mais informações sobre o Seaborn, recomenda-se consultar a documentação oficial, que inclui uma galeria de exemplos e tutoriais detalhados.

Este material abordou apenas uma introdução à biblioteca. Nos próximos capítulos, exploraremos funcionalidades mais avançadas e aplicações específicas do Seaborn para visualizações estatísticas e análise de dados.

02. Plots de distribuição

A visualização de dados é uma etapa fundamental na análise exploratória, pois permite identificar padrões, tendências e possíveis outliers de forma intuitiva. Quando se trata de explorar a distribuição de variáveis, o Seaborn é uma das bibliotecas mais poderosas do ecossistema Python, oferecendo diversas opções de gráficos que facilitam essa tarefa.

Neste capítulo, vamos explorar diferentes tipos de gráficos de distribuição disponíveis no Seaborn, como histogramas, gráficos de densidade, gráficos conjuntos e pair plots. Esses gráficos ajudam a entender melhor a distribuição dos dados e como diferentes variáveis podem estar relacionadas.

Importando a Biblioteca Seaborn

Para começar a utilizar o Seaborn, é necessário importá-lo. O padrão é importá-lo como `sns`, embora isso não seja obrigatório.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Garante que os plots sejam exibidos corretamente em notebooks Jupyter
%matplotlib inline
```

Carregando Conjunto de Dados

O Seaborn possui alguns conjuntos de dados internos que podem ser utilizados para exploração e criação de visualizações. Um dos conjuntos de dados disponíveis é o `tips`, que contém informações sobre gorjetas em um restaurante fictício.

```
tips = sns.load_dataset("tips")
print(tips.head())
```

Esse conjunto de dados inclui informações como:

- `total_bill`: Valor total da conta.
- `tip`: Valor da gorjeta.
- `sex`: Gênero do cliente.
- `smoker`: Se o cliente é fumante ou não.
- `day`: Dia da semana.
- `time`: Período (almoço ou jantar).
- `size`: Quantidade de pessoas na mesa.

Gráficos de Distribuição

Os gráficos de distribuição são ferramentas essenciais na análise exploratória de dados, pois permitem entender como os valores de uma variável se espalham ao longo de um intervalo. Eles ajudam a identificar padrões, detectar outliers e verificar a simetria ou assimetria da distribuição dos dados.

Histograma com `sns.histplot`

O `histplot` é uma das formas mais simples de visualizar distribuições de dados.

```
sns.histplot(tips["total_bill"], kde=True)
plt.show()
```

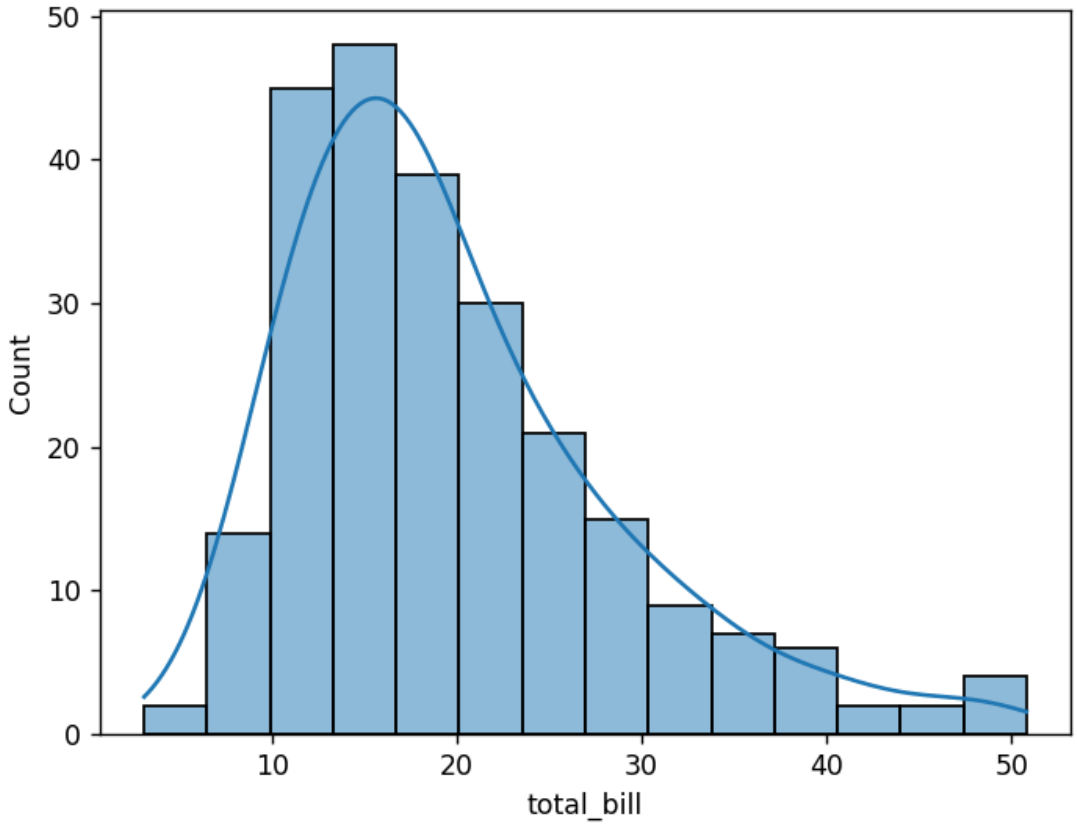

Figure 1: Ilustração do gráfico

No exemplo abaixo, além do histograma, é gerada uma curva KDE (Kernel Density Estimate), que estima a distribuição da variável.

- O KDE pode ser desativado com `kde=False`.
- A quantidade de "bins" pode ser ajustada com o parâmetro `bins`.

```
sns.histplot(tips["total_bill"], kde=False, bins=30)
plt.show()
```

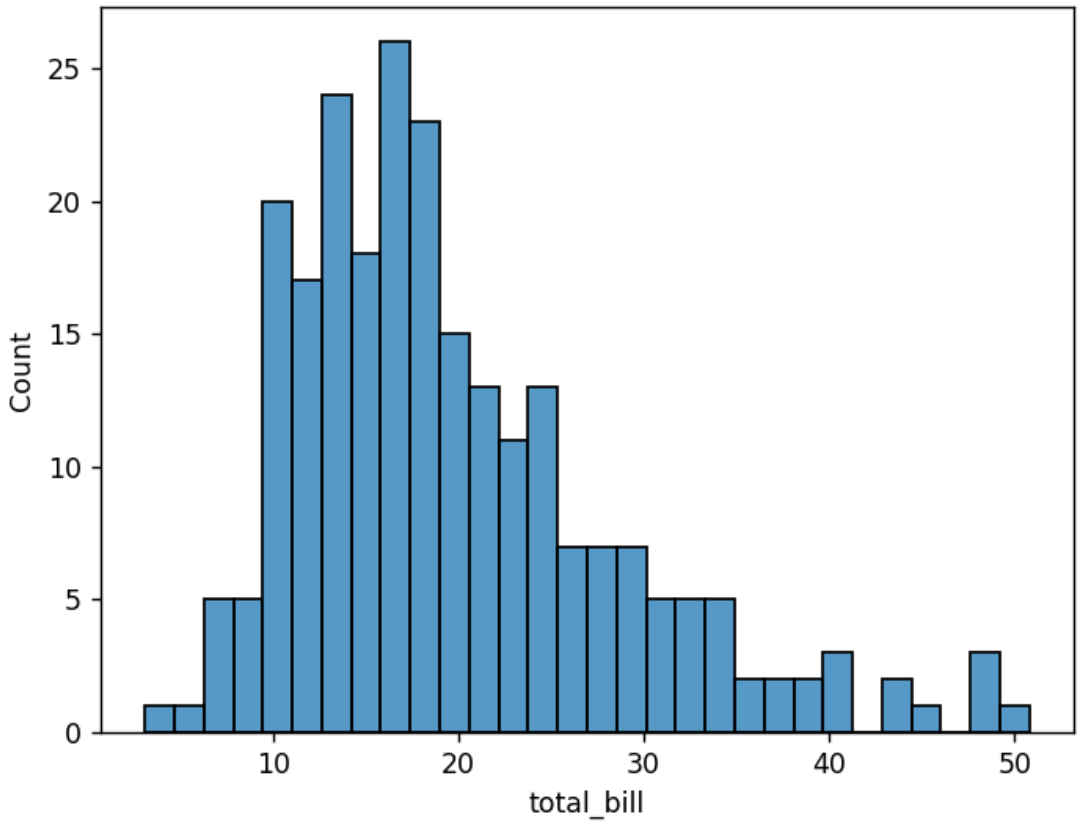

Figure 2: Ilustração do gráfico

Gráfico de Distribuição Conjunta com `sns.jointplot`

O `jointplot` permite visualizar a relação entre duas variáveis simultaneamente, combinando histogramas e dispersão de pontos.

```
sns.jointplot(x="total_bill", y="tip", data=tips, kind="scatter")
plt.show()
```

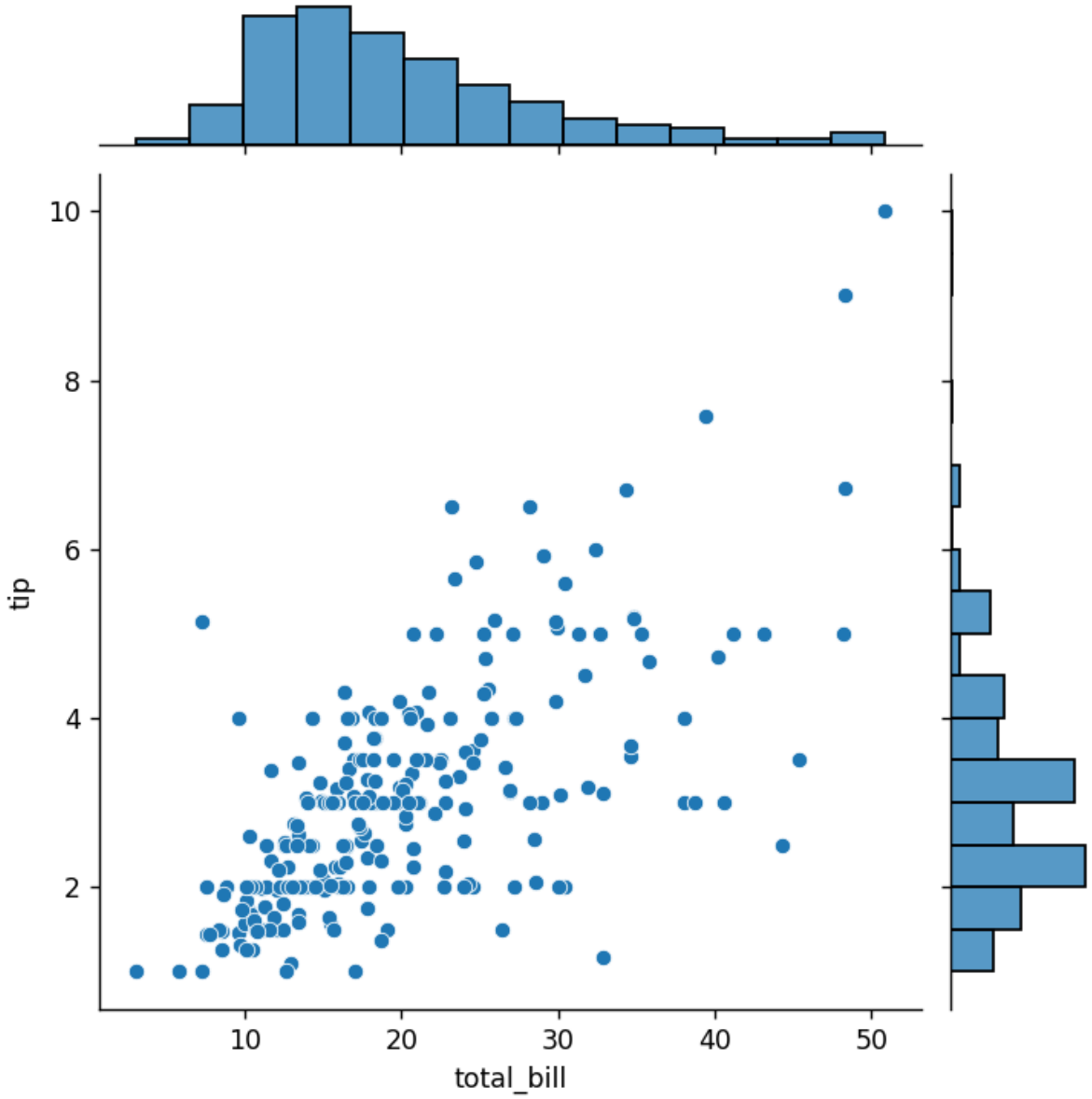

Figure 3: Ilustração do gráfico

Outros tipos de `jointplot` incluem:

- `kind="reg"`: Adiciona uma linha de regressão.
- `kind="hex"`: Usa um mapa de calor hexagonal para mostrar a densidade dos dados.

```
sns.jointplot(x="total_bill", y="tip", data=tips, kind="reg")
plt.show()
```

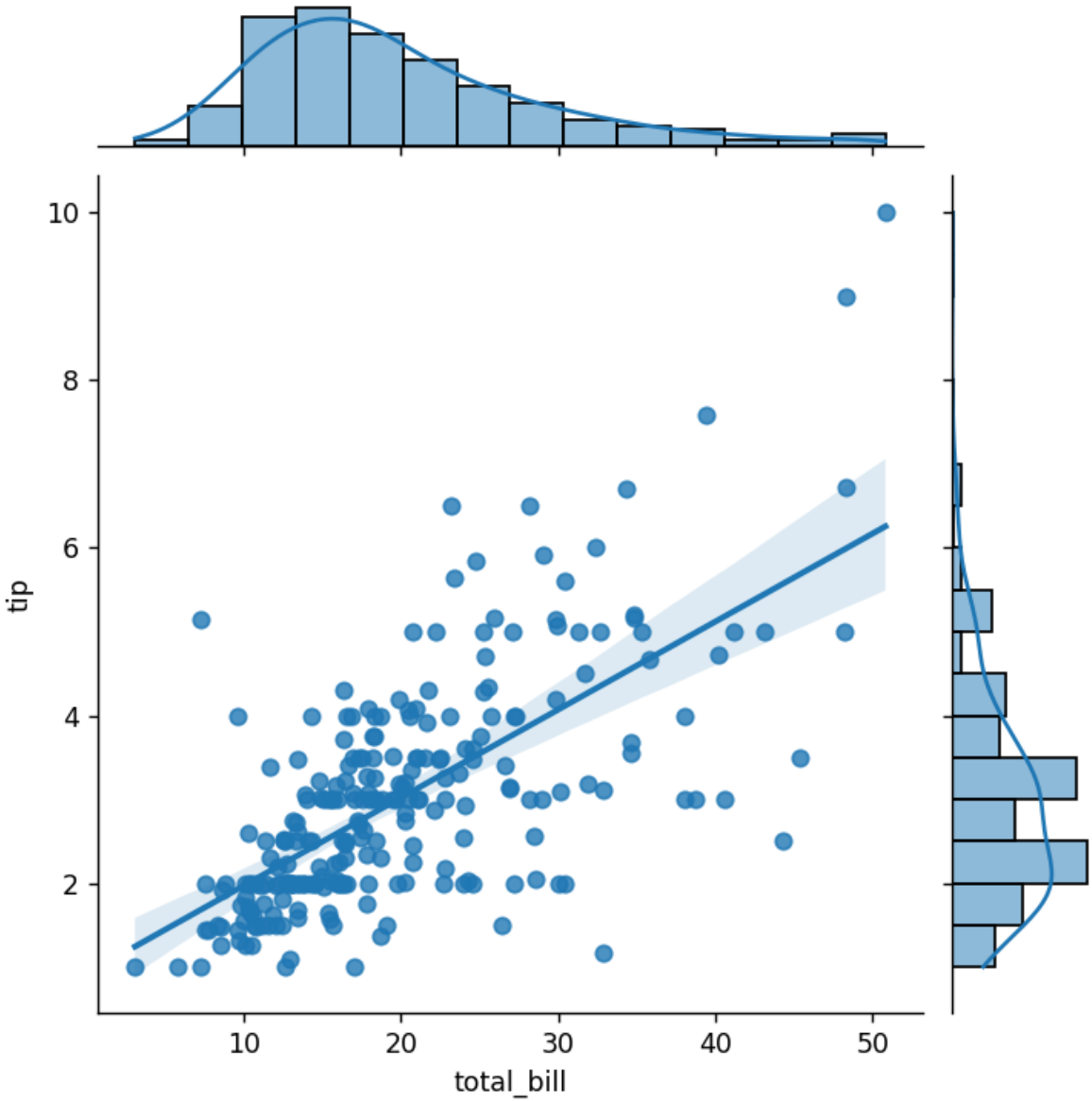

Figure 4: Ilustração do gráfico

Gráfico de Correlação com `sns.pairplot`

O `pairplot` cria uma matriz de scatter plots entre todas as variáveis numéricas do conjunto de dados.

```
sns.pairplot(tips)
plt.show()
```

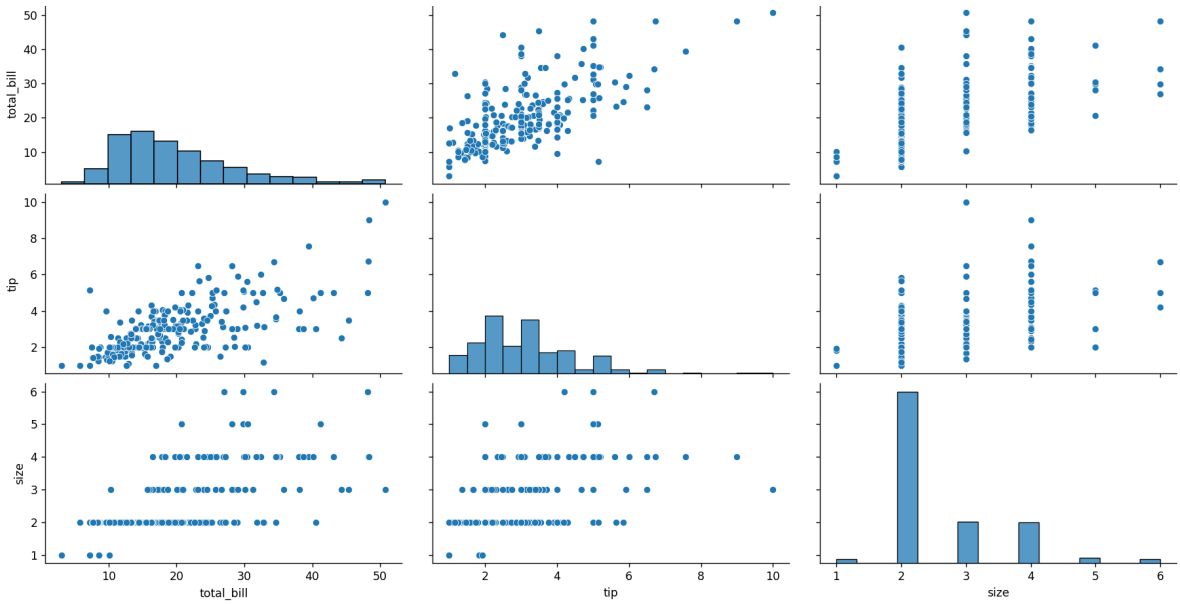

Figure 5: Ilustração do gráfico

Para segmentar os dados por uma variável categórica, utilize o parâmetro hue:

```
sns.pairplot(tips, hue="sex", palette="coolwarm")
plt.show()
```

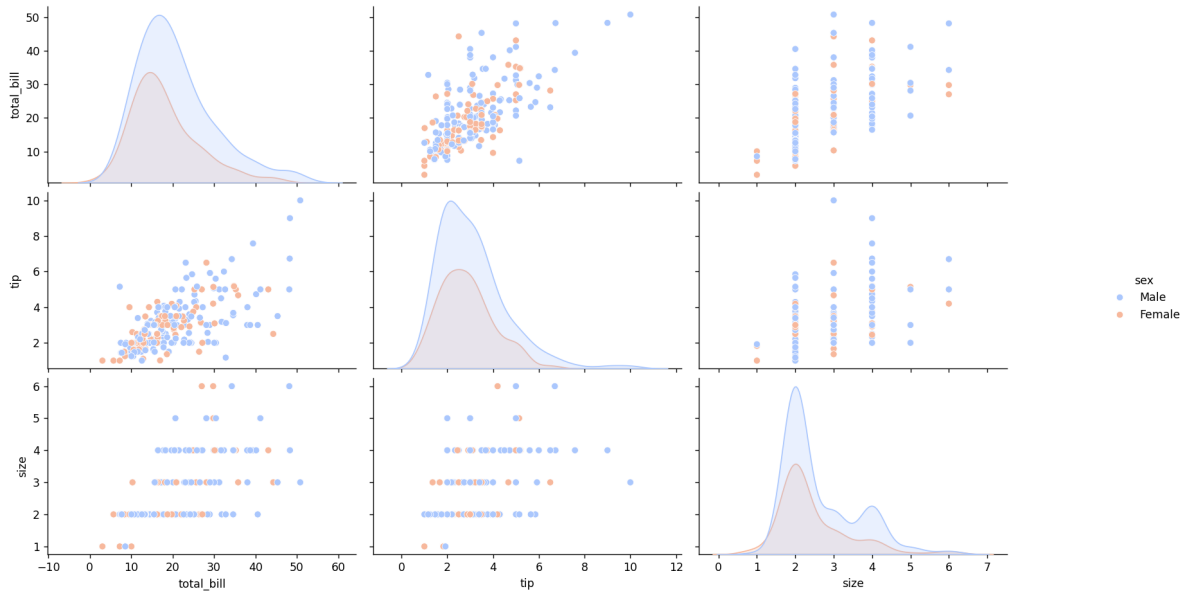

Figure 6: Ilustração do gráfico

Isso permite visualizar como os dados se distribuem por categorias, como gênero ou período da

refeição.

Rug Plot com `sns.rugplot`

O `rugplot` mostra a distribuição dos dados ao longo de um eixo.

```
sns.rugplot(x=tips["total_bill"])
plt.show()
```

Geralmente, ele é combinado com o KDE plot:

```
sns.kdeplot(x=tips["total_bill"])
sns.rugplot(x=tips["total_bill"], height=0.1)
plt.show()
```

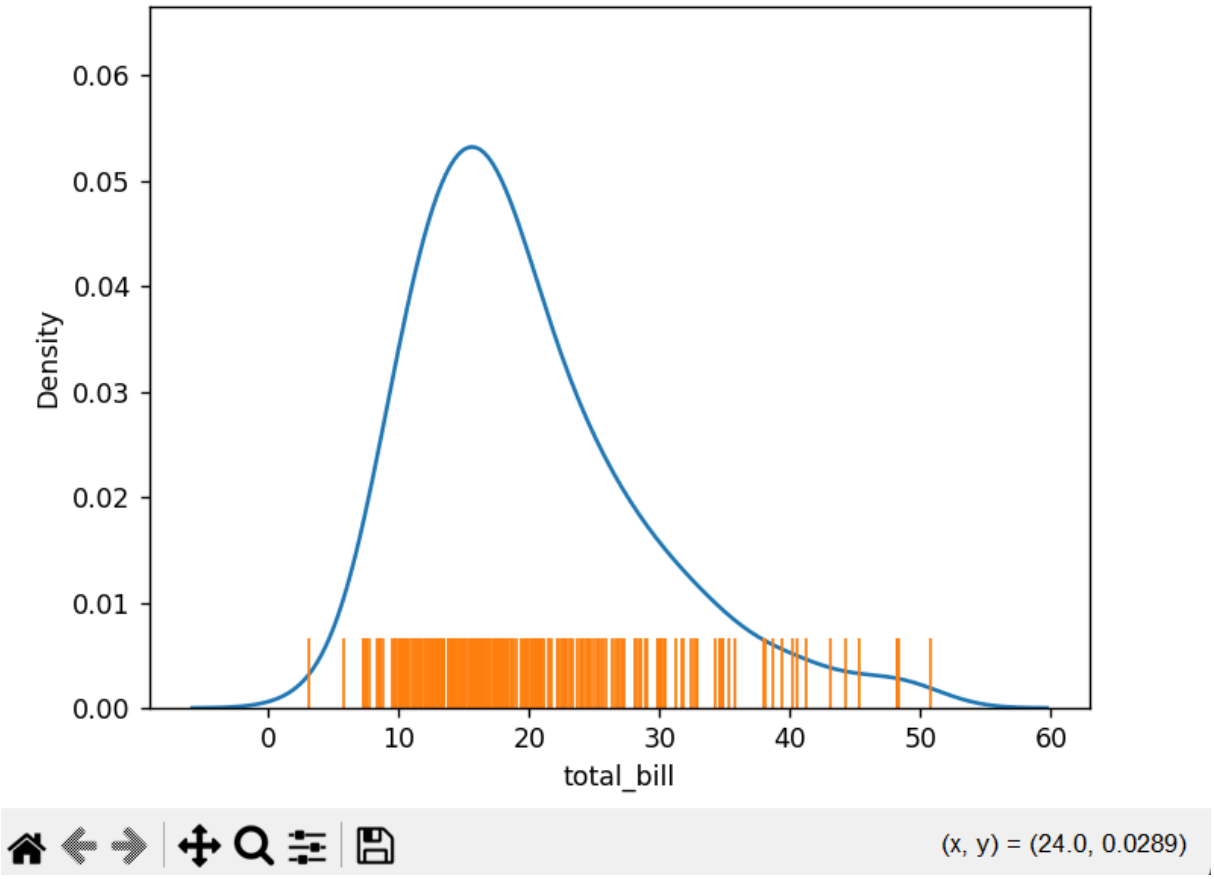

Figure 7: Ilustração do gráfico

Conclusão

Neste capítulo, foram abordados diferentes tipos de gráficos de distribuição de dados no Seaborn. Cada um deles possui funcionalidades específicas que permitem compreender melhor a estrutura dos dados e identificar padrões relevantes.

Os próximos capítulos abordarão outros tipos de visualizações, como gráficos categóricos e análise de regressão.

03. Plots categóricos

No capítulo anterior, exploramos gráficos de distribuição, que são ideais para analisar variáveis contínuas. No entanto, quando lidamos com variáveis categóricas, como gênero, dia da semana ou status de fumante, precisamos de gráficos específicos para representar essa informação. O Seaborn nos oferece diversas opções para isso, incluindo `barplot`, `countplot`, `boxplot`, `violinplot` e `swarmplot`.

Gráficos de Barras: `barplot` e `countplot`

O `barplot` permite visualizar a relação entre uma variável categórica e uma variável numérica, exibindo a média (ou outra métrica estatística) dos valores numéricos para cada categoria.

```
sns.barplot(x="sex", y="total_bill", data=tips)
plt.show()
```

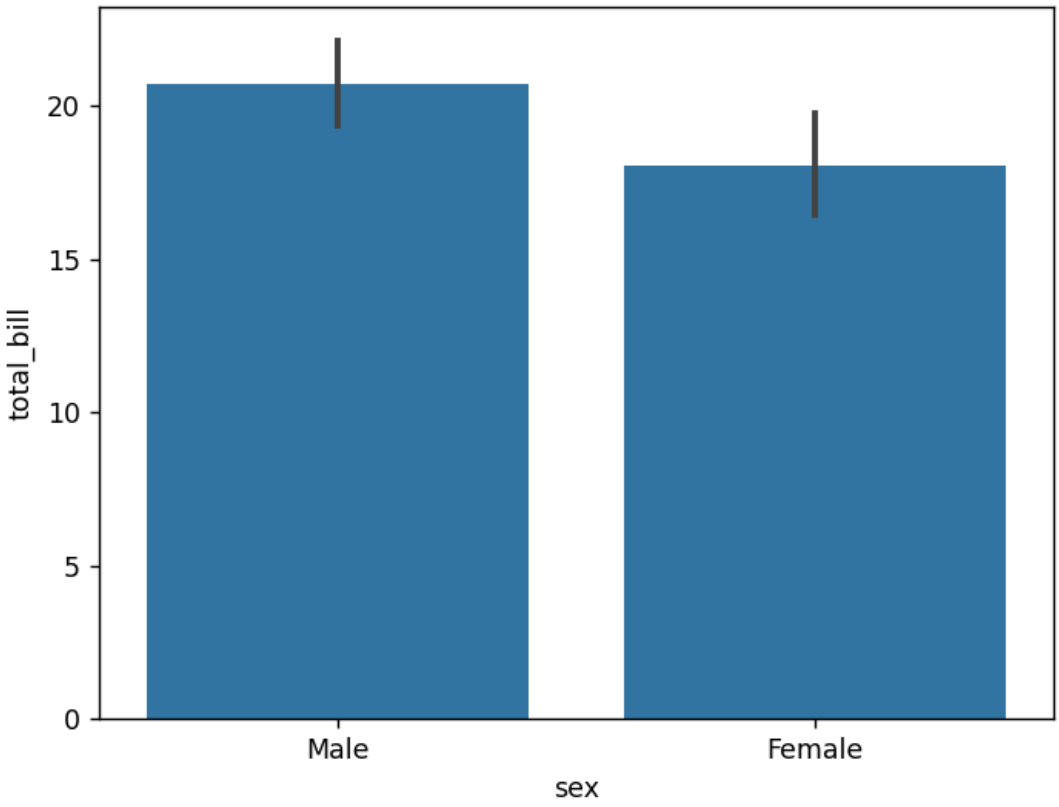

Figure 8: Ilustração do gráfico

A linha preta nos topos das barras representa o intervalo de confiança dos dados.

Já o `countplot` exibe a contagem de ocorrências de cada categoria, sem necessidade de uma variável numérica.

```
sns.countplot(x="sex", data=tips)
plt.show()
```

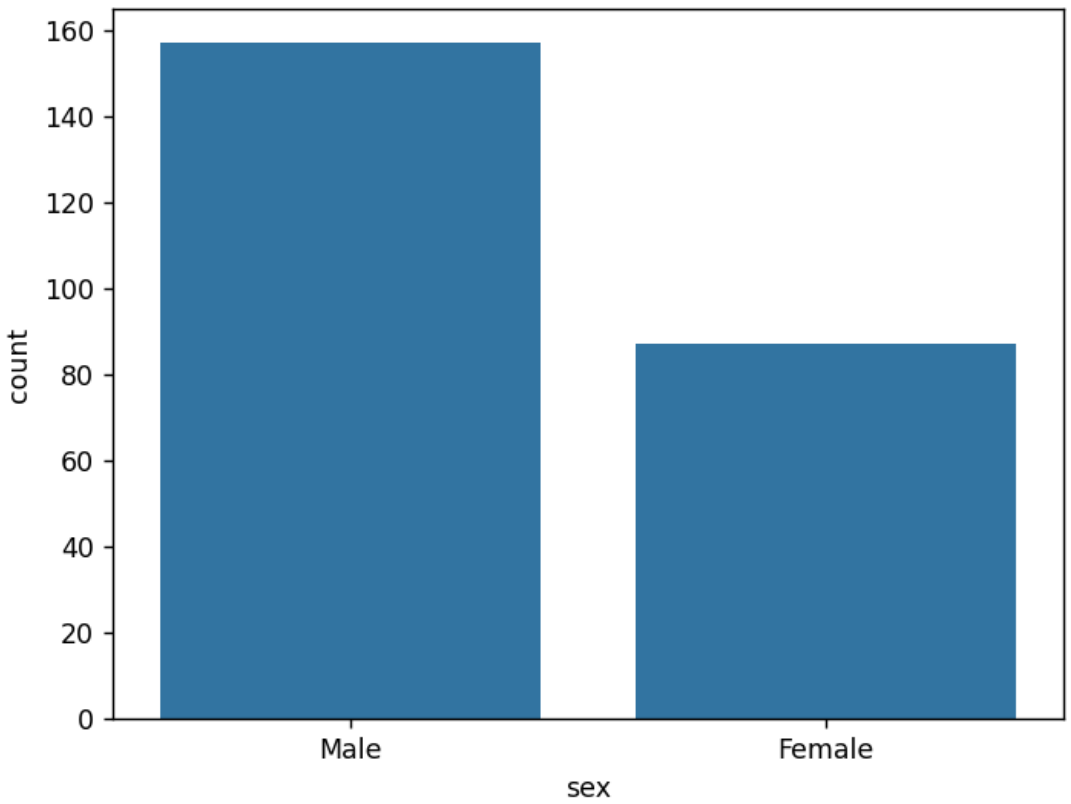

Figure 9: Ilustração do gráfico

Boxplot

O `boxplot` é útil para visualizar a distribuição de uma variável numérica dentro de categorias.

```
sns.boxplot(x="day", y="total_bill", data=tips, palette="rainbow")
plt.show()
```

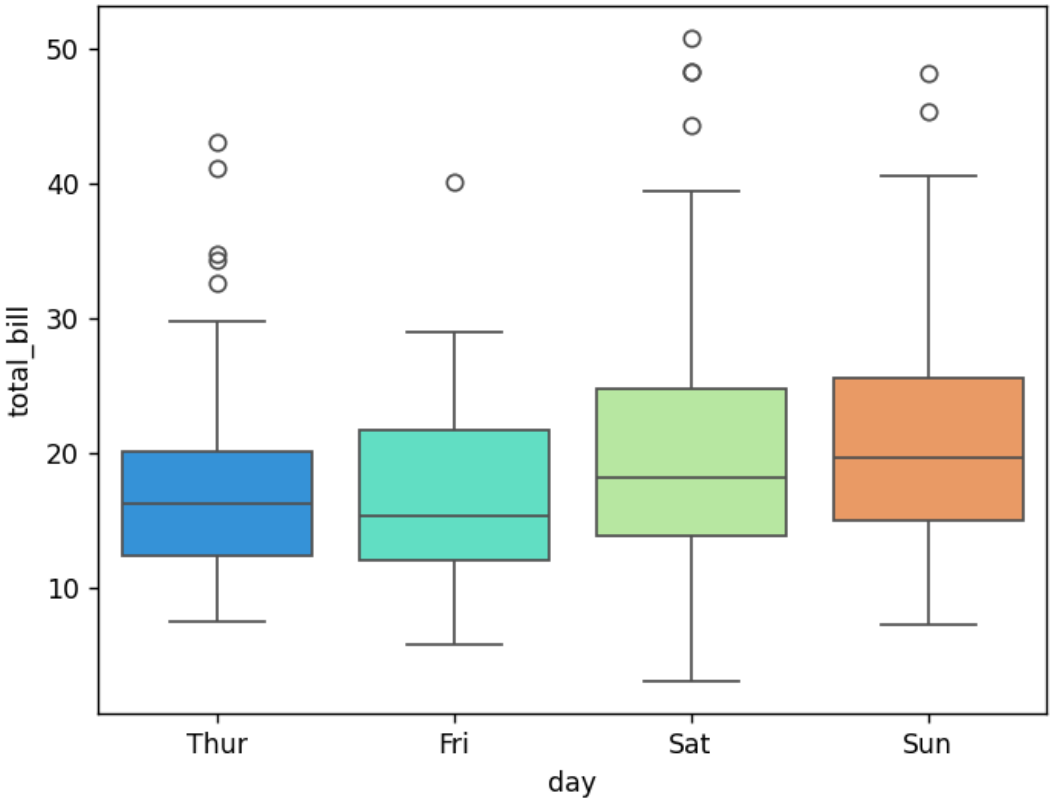

Figure 10: Ilustração do gráfico

O boxplot exibe os quartis, a mediana e possíveis outliers, permitindo uma análise detalhada da dispersão dos dados.

Podemos também dividir os dados por uma terceira variável categórica usando hue:

```
sns.boxplot(x="day", y="total_bill", hue="sex", data=tips)
plt.show()
```

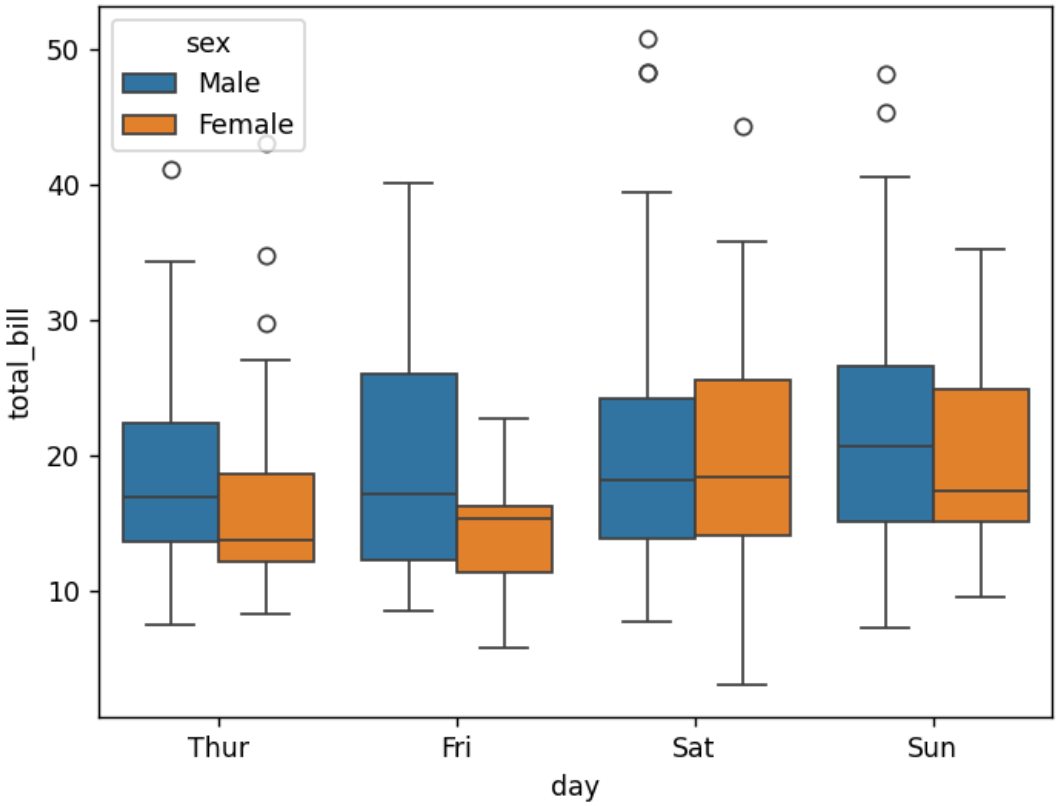

Figure 11: Ilustração do gráfico

Violinplot

O `violinplot` combina características do `boxplot` com estimativas de densidade para mostrar a distribuição dos dados.

```
sns.violinplot(x="day", y="total_bill", data=tips, hue="sex", split=True)
plt.show()
```

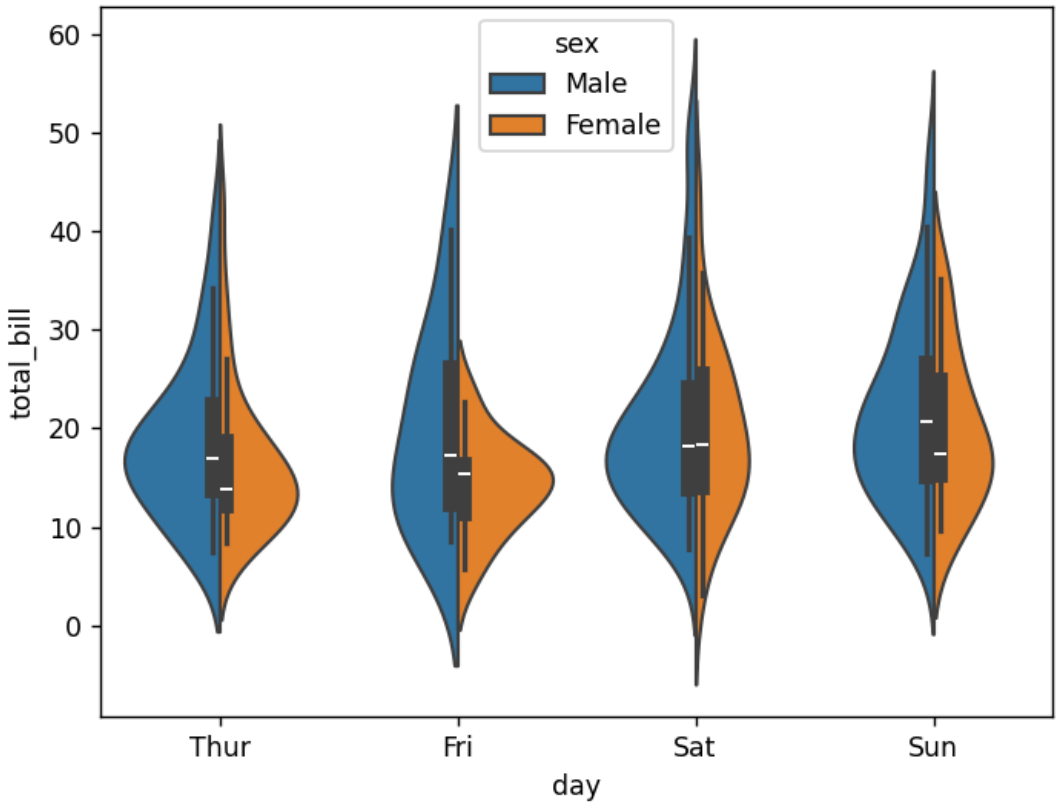

Figure 12: Ilustração do gráfico

Swarmplot

O `swarmplot` mostra todos os pontos individuais dos dados, distribuindo-os para evitar sobreposição.

```
sns.swarmplot(x="day", y="total_bill", data=tips)
plt.show()
```

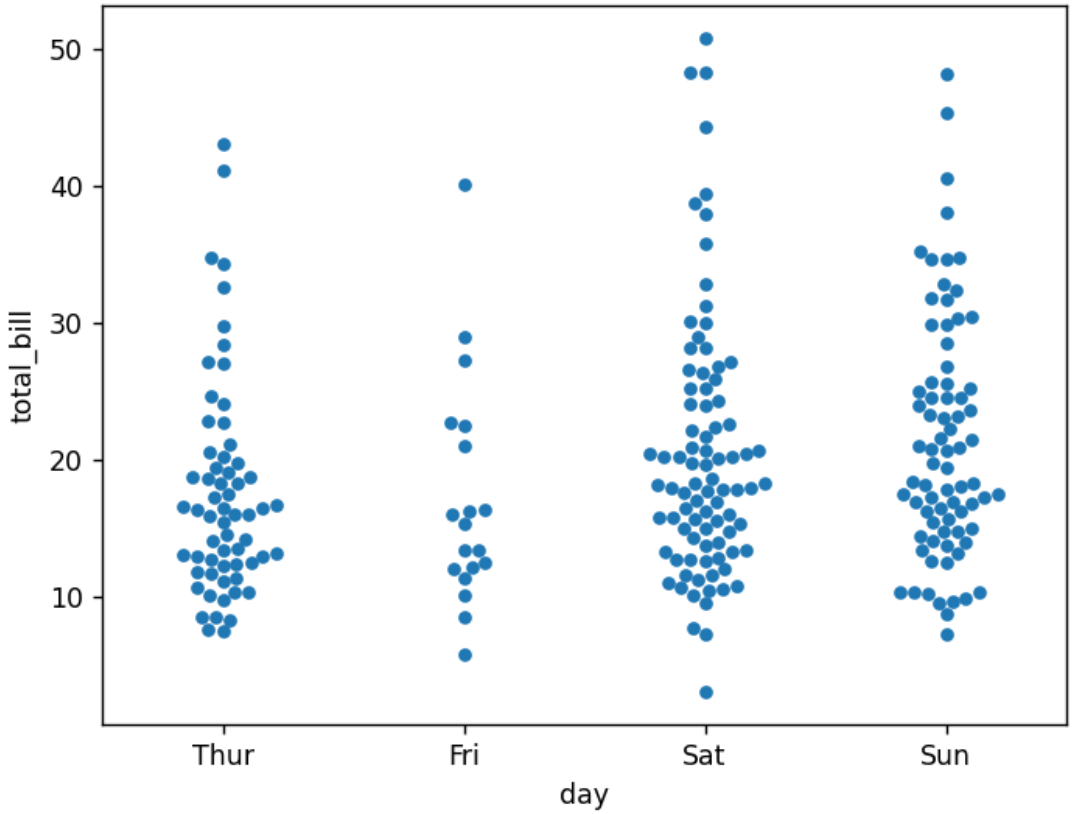

Figure 13: Ilustração do gráfico

O `swarmplot` pode ser combinado com o `boxplot` para enriquecer a visualização:

```
sns.boxplot(x="day", y="total_bill", data=tips)
sns.swarmplot(x="day", y="total_bill", data=tips, color="black")
plt.show()
```

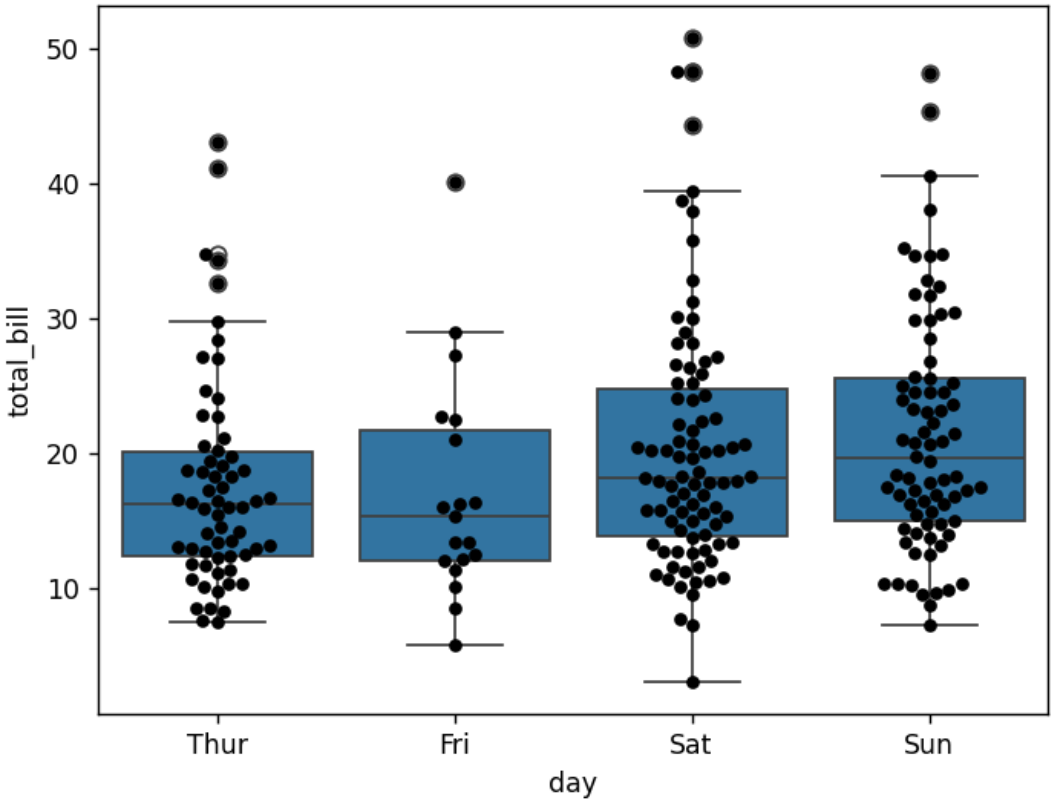

Figure 14: Ilustração do gráfico

Conclusão

Os gráficos categóricos são essenciais para entender padrões em variáveis qualitativas e sua relação com variáveis numéricas. No próximo capítulo, exploraremos os gráficos de regressão, que nos ajudam a visualizar relações entre variáveis numéricas.

04. Plots de regressão

Neste capítulo, vamos explorar os plots de regressão que o Seaborn nos oferece para representar graficamente relações lineares entre duas variáveis do nosso conjunto de dados. Como exemplo, utilizaremos o dataset *tips* e analisaremos a relação entre variáveis utilizando os gráficos apropriados.

O que é o LMPlot?

O *LMPlot* (*Linear Model Plot*) é um dos gráficos mais utilizados para visualização de regressão no Seaborn. Ele permite criar gráficos de dispersão com linhas de tendência baseadas em modelos de regressão linear.

Para utilizá-lo, especificamos os eixos X e Y, juntamente com o conjunto de dados:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Carregando o dataset
dados = sns.load_dataset("tips")

# Criando o LMPlot
sns.lmplot(x="total_bill", y="tip", data=dados)
plt.show()
```

O gráfico gerado exibe a relação entre o total da conta (*total_bill*) e a gorjeta (*tip*), indicando uma tendência linear.

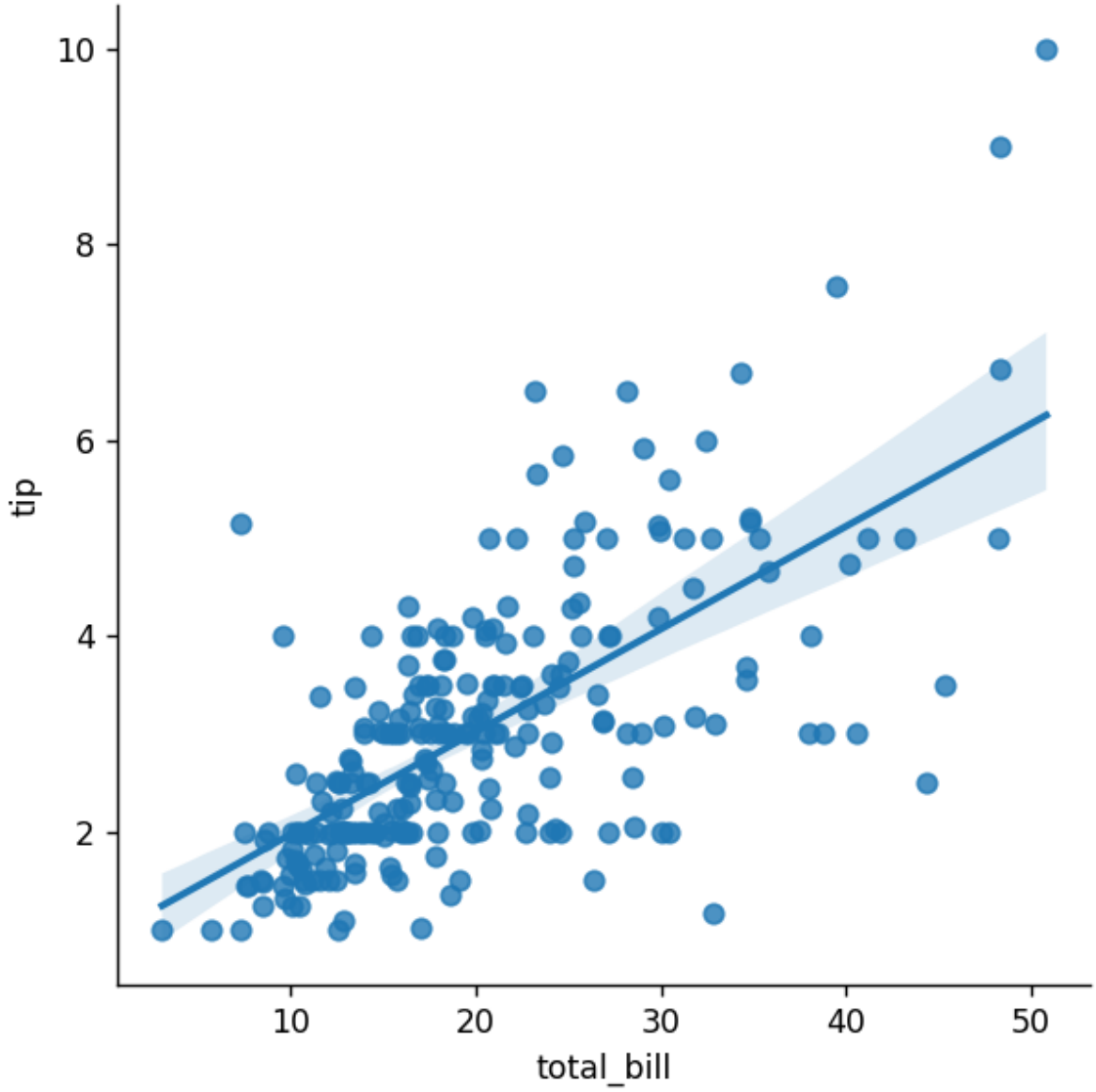

Figure 15: Ilustração do gráfico

Personalização com o parâmetro hue

Podemos segmentar os dados por categorias, como gênero, utilizando o parâmetro hue:

```
sns.lmplot(x="total_bill", y="tip", hue="sex", data=dados)
plt.show()
```

Isso nos permite visualizar separadamente a relação linear para homens e mulheres. Em nosso exem-

plo, a tendência é semelhante para ambos, com pequenas variações dentro da margem de incerteza do modelo.

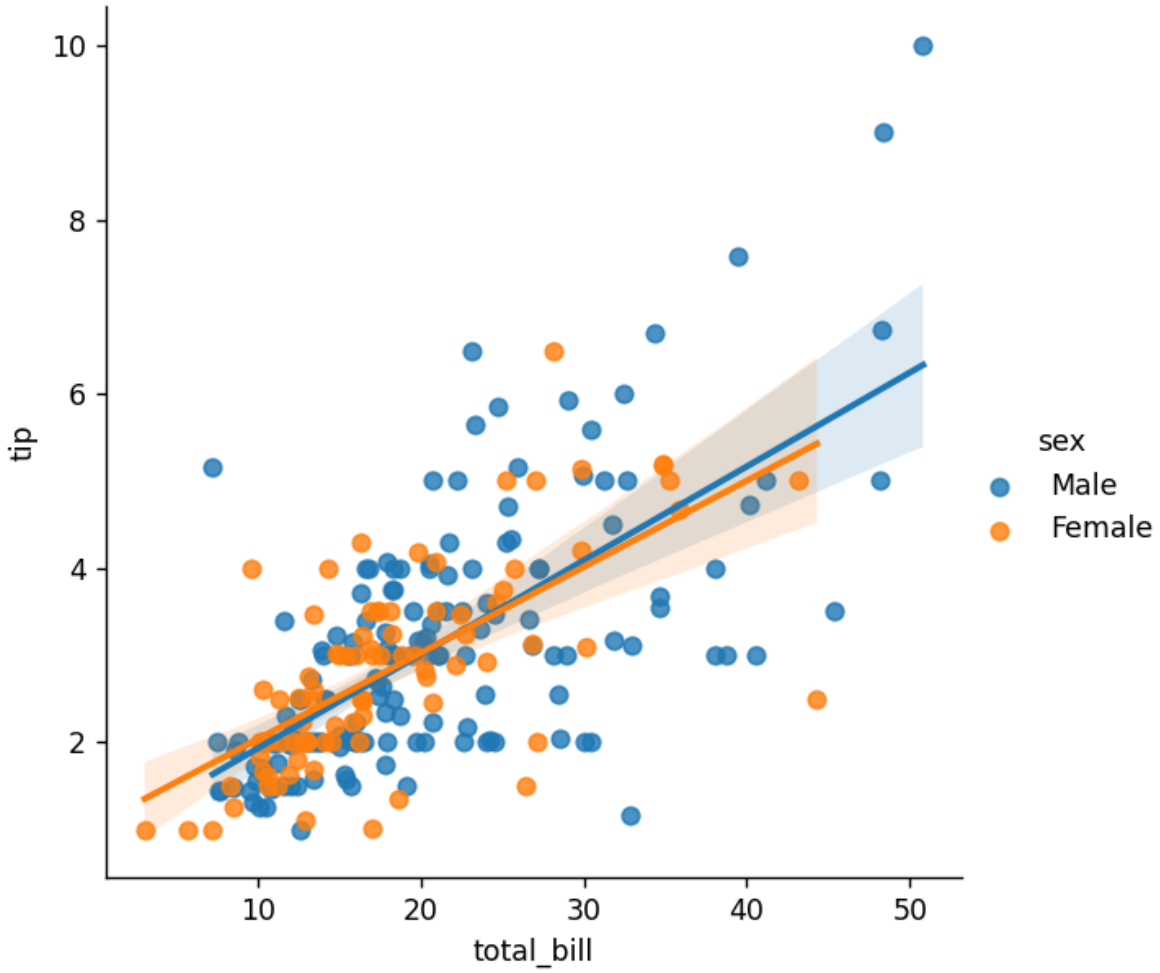

Figure 16: Ilustração do gráfico

Ajustando Marcadores

Podemos personalizar os marcadores dos pontos no gráfico utilizando o parâmetro `markers`:

```
sns.lmplot(x="total_bill", y="tip", hue="sex", data=dados, markers=["o", "v"])
plt.show()
```

Neste caso, utilizamos um círculo (`"o"`) para homens e um triângulo invertido (`"v"`) para mulheres.

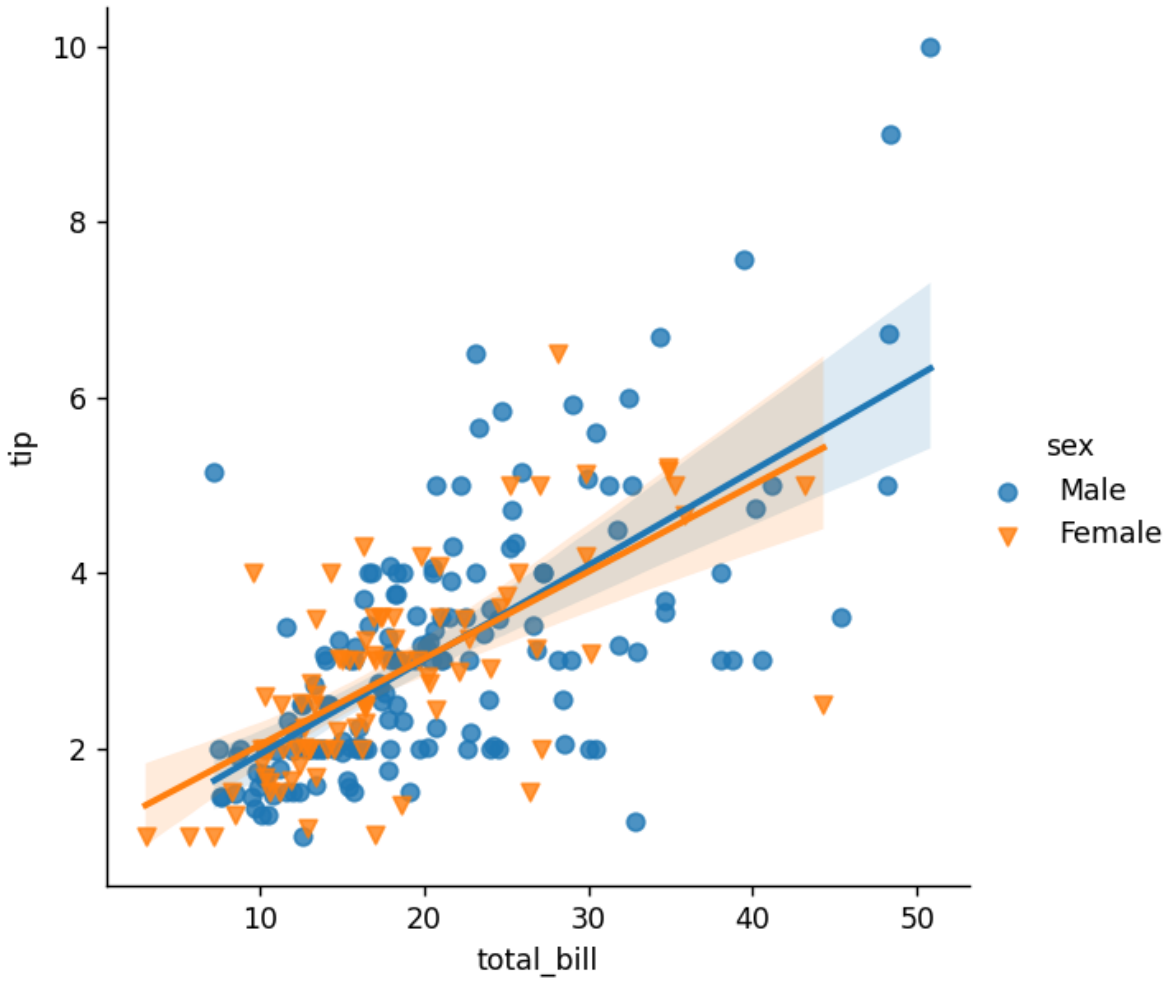

Figure 17: Ilustração do gráfico

Modificando o Tamanho dos Pontos

Para aumentar o tamanho dos pontos no gráfico, usamos o parâmetro `scatter_kws`:

```
sns.lmplot(x="total_bill", y="tip", hue="sex", data=dados, scatter_kws={"s": 100})
plt.show()
```

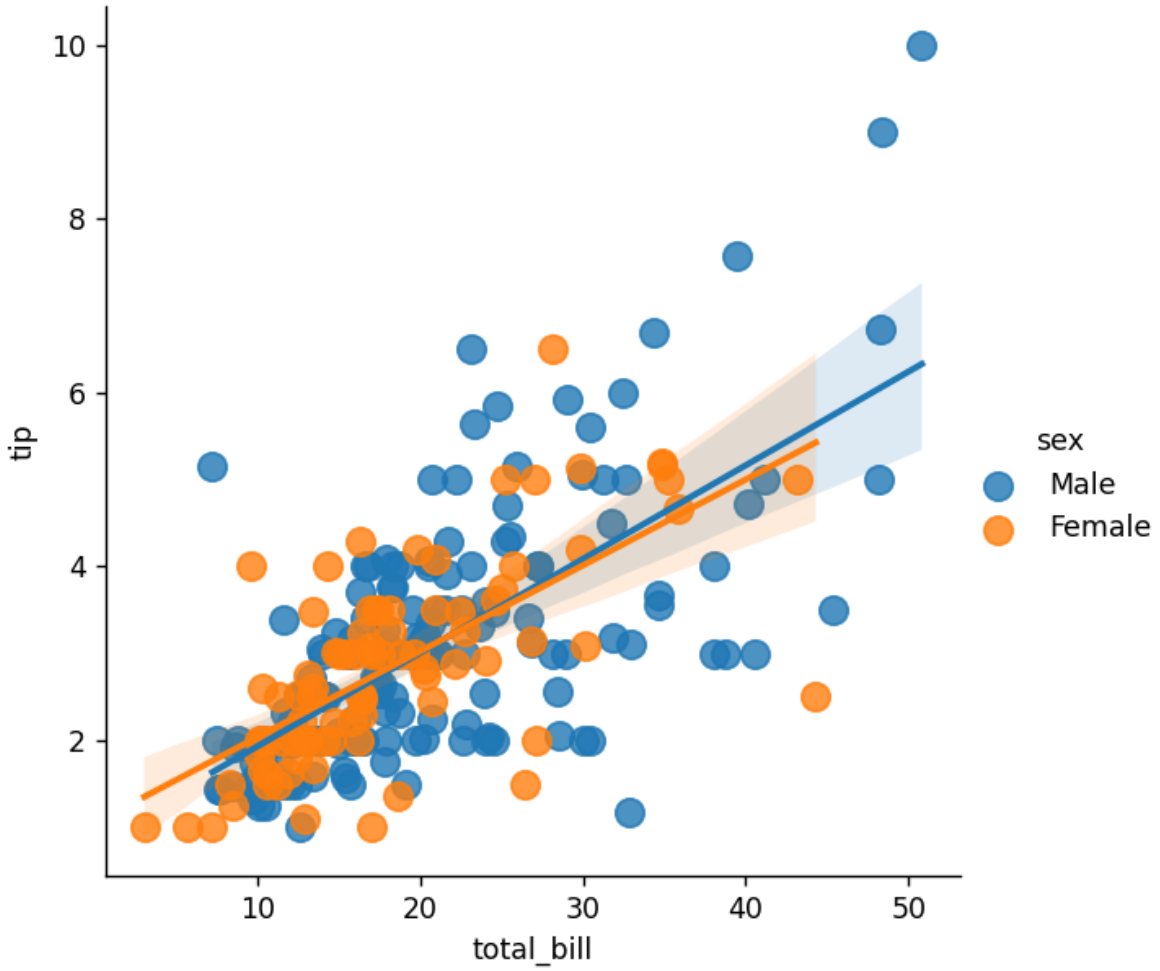

Figure 18: Ilustração do gráfico

Separando os Gráficos em Colunas

Podemos dividir os gráficos de regressão em diferentes colunas com o parâmetro `col`. No exemplo abaixo, segmentamos os dados pelo período (*time*), separando almoço e jantar:

```
sns.lmplot(x="total_bill", y="tip", hue="sex", col="time", data=dados)
plt.show()
```

O Seaborn cria dois gráficos distintos, um para o almoço e outro para o jantar, permitindo comparação direta entre os dois períodos.

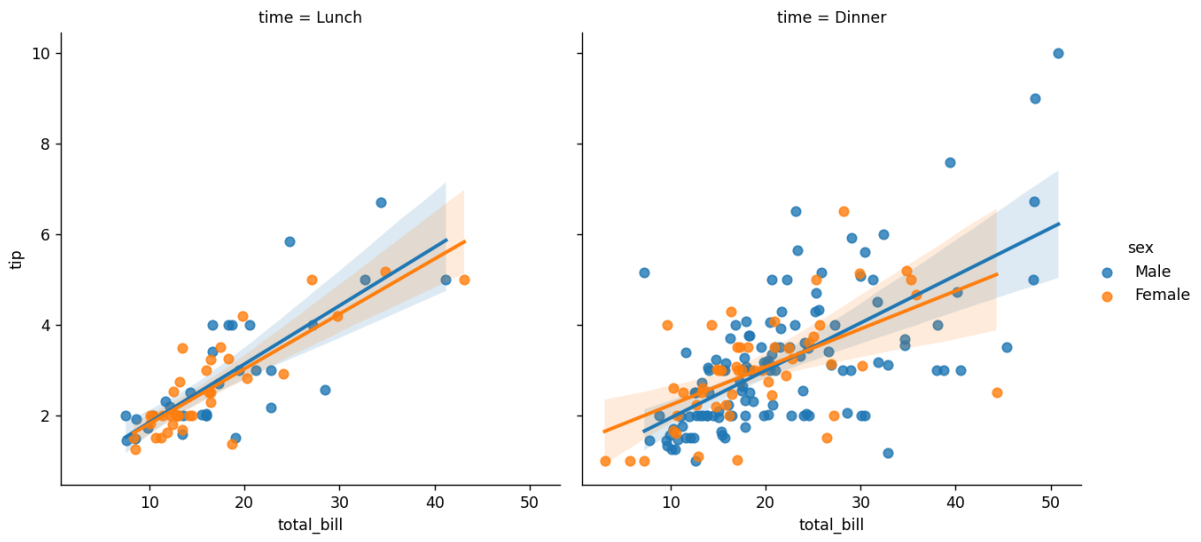

Figure 19: Ilustração do gráfico

Segmentação por Linhas

Da mesma forma, podemos dividir os dados em diferentes linhas utilizando `row`:

```
sns.lmplot(x="total_bill", y="tip", hue="sex", row="day", data=dados)
plt.show()
```

Isso gera gráficos individuais para cada dia da semana presente no dataset.

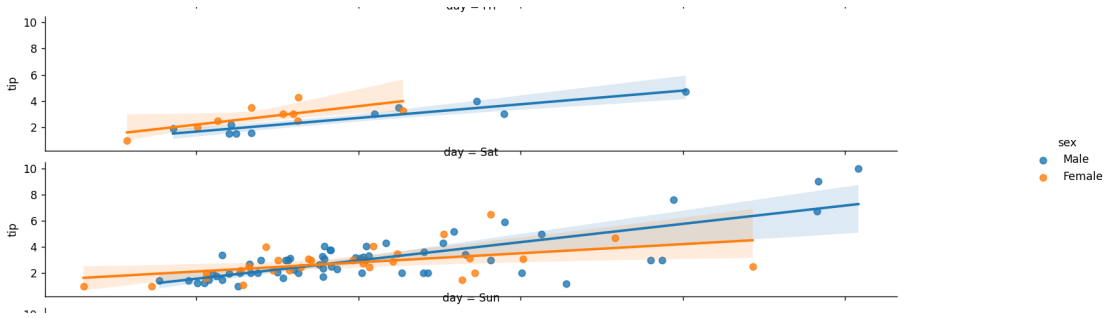

Figure 20: Ilustração do gráfico

Ajustando Dimensões do Gráfico

Podemos modificar as dimensões do gráfico com os parâmetros `aspect` e `size`:

```
sns.lmplot(x="total_bill", y="tip", data=dados, aspect=1.6, height=8)
plt.show()
```

O parâmetro `aspect` altera a proporção entre largura e altura do gráfico, enquanto `height` define sua altura total.

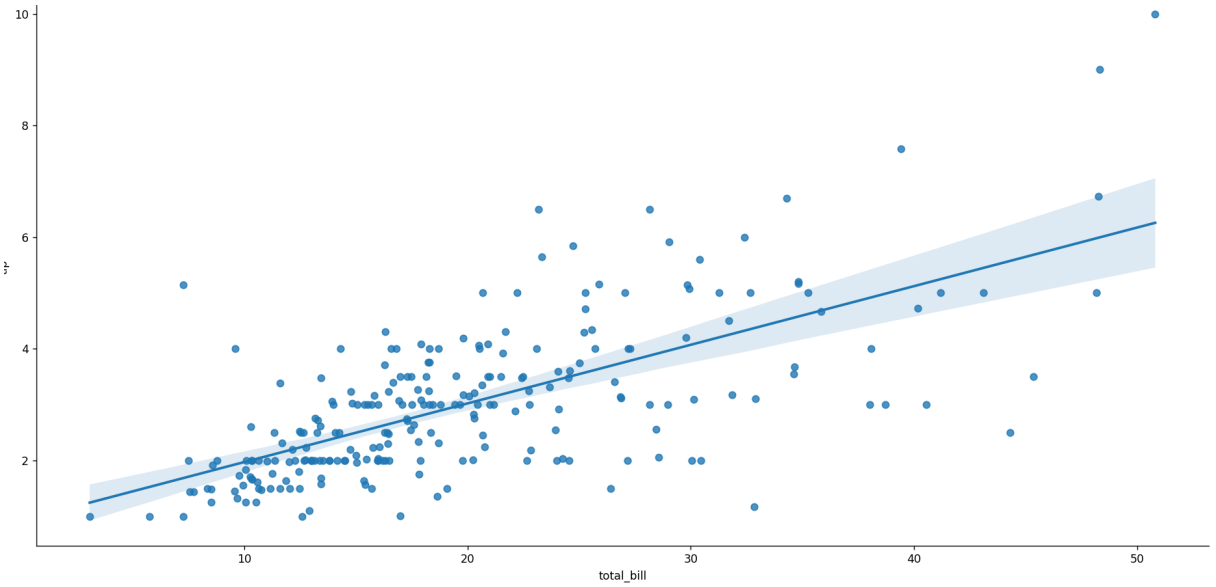

Figure 21: Ilustração do gráfico

Conclusão

O *LMPlot* é uma ferramenta poderosa para análise de relações lineares entre variáveis. Ele permite uma visualização clara e segmentada dos dados, facilitando a compreensão de padrões e tendências.

Recomenda-se explorar os diferentes parâmetros mencionados para encontrar a melhor configuração para seu caso específico.

Na próxima aula, abordaremos os estilos de visualização no Seaborn!

05. Plots Matriciais

Dando continuidade ao nosso estudo de Seaborn, agora entraremos na parte de plots matriciais, ou os conhecidos heatmaps. Para isso, utilizaremos um conjunto de dados do Seaborn chamado `flights`, que contém informações sobre a quantidade de voos realizados por uma companhia aérea hipotética ao longo de diversos meses e anos. Esse conjunto de dados possui 144 linhas e será usado para visualizar padrões e tendências ao longo do tempo.

O plot mais importante do Seaborn para visualizar informações em três dimensões é o `sns.heatmap`. Esse método permite a criação de um mapa de calor contendo três eixos:

- **Eixo X:** Representa um conjunto categórico, como os anos.
- **Eixo Y:** Representa outro conjunto categórico, como os meses.
- **Eixo Z:** Representado pela cor, que indica a quantidade de voos.

Transformação dos Dados

Para criar um mapa de calor, os dados precisam estar em formato matricial. Como temos repetição de meses ao longo de vários anos, utilizamos a função `pivot_table` do Pandas para reorganizar os dados:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Carregar dados
flights = sns.load_dataset("flights")

# Reorganizar os dados
flights_matrix = flights.pivot_table(index="year", columns="month", values="passengers")
```

Essa operação transforma os dados em uma tabela onde os anos são as linhas, os meses são as colunas e os valores representam a quantidade de passageiros.

Criando um Heatmap

Agora podemos passar esse novo DataFrame para a função `sns.heatmap`:

```
plt.figure(figsize=(12, 6))
sns.heatmap(flights_matrix, cmap="coolwarm", annot=True, fmt=".0f")
plt.title("Mapa de Calor da Quantidade de Voos")
plt.show()
```

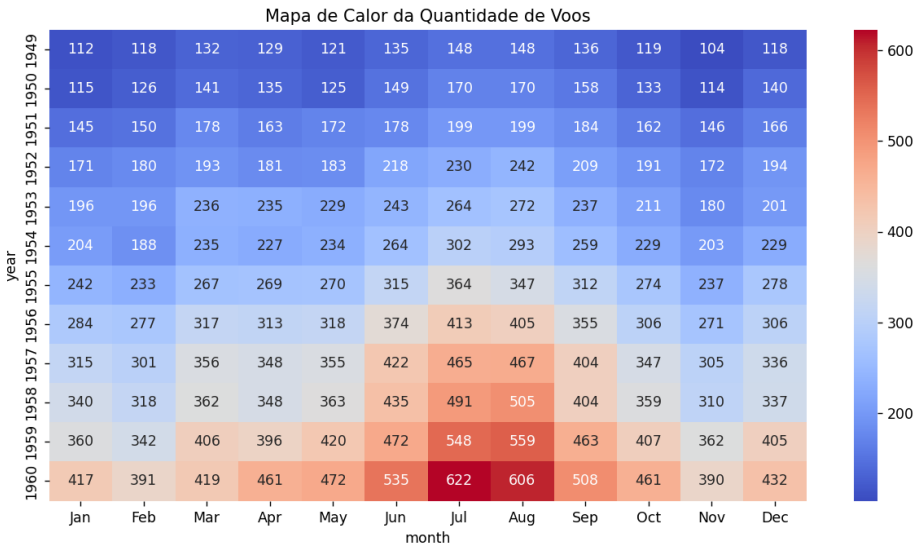

Figure 22: Ilustração do gráfico

Ajustando o Tamanho do Gráfico

Para melhorar a visualização, podemos aumentar o tamanho do gráfico utilizando `plt.subplots()`:

```
fig, ax = plt.subplots(figsize=(12, 6))
sns.heatmap(flights_matrix, cmap="coolwarm", annot=True, fmt=".0f")
plt.title("Mapa de Calor da Quantidade de Voos")
plt.show()
```

Isso facilita a manipulação do gráfico utilizando as ferramentas do `matplotlib`.

Personalização do Heatmap

Podemos ajustar o estilo e a apresentação do gráfico utilizando diferentes parâmetros:

- `annot=True`: Exibe os valores dentro das células.
- `fmt=".0f"`: Define a formatação dos números para valores inteiros.
- `linewidths=1`: Adiciona separação entre as células.
- `cmap="magma"`: Define uma paleta de cores alternativa.

Exemplo de código com essas personalizações:

```
plt.figure(figsize=(12, 6))
sns.heatmap(flights_matrix, cmap="magma", annot=True, fmt=".0f", linewidths=1)
plt.title("Mapa de Calor da Quantidade de Voos")
plt.show()
```

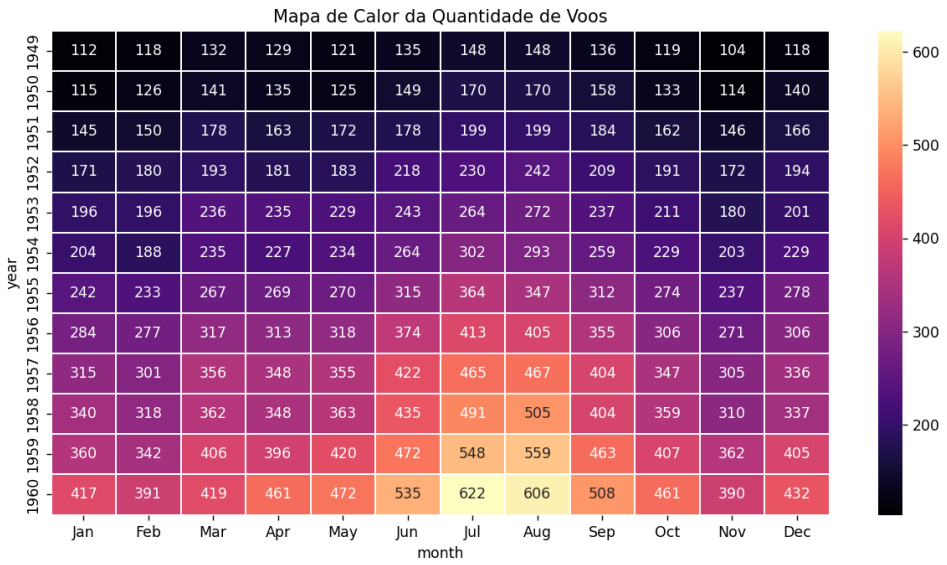

Figure 23: Ilustração do gráfico

Cluster Map

Outro gráfico interessante é o `cluster` map, que reorganiza os dados para agrupar valores similares:

```
sns.clustermap(flights_matrix, cmap="coolwarm", standard_scale=1)
plt.title("Cluster Map da Quantidade de Voos")
plt.show()
```

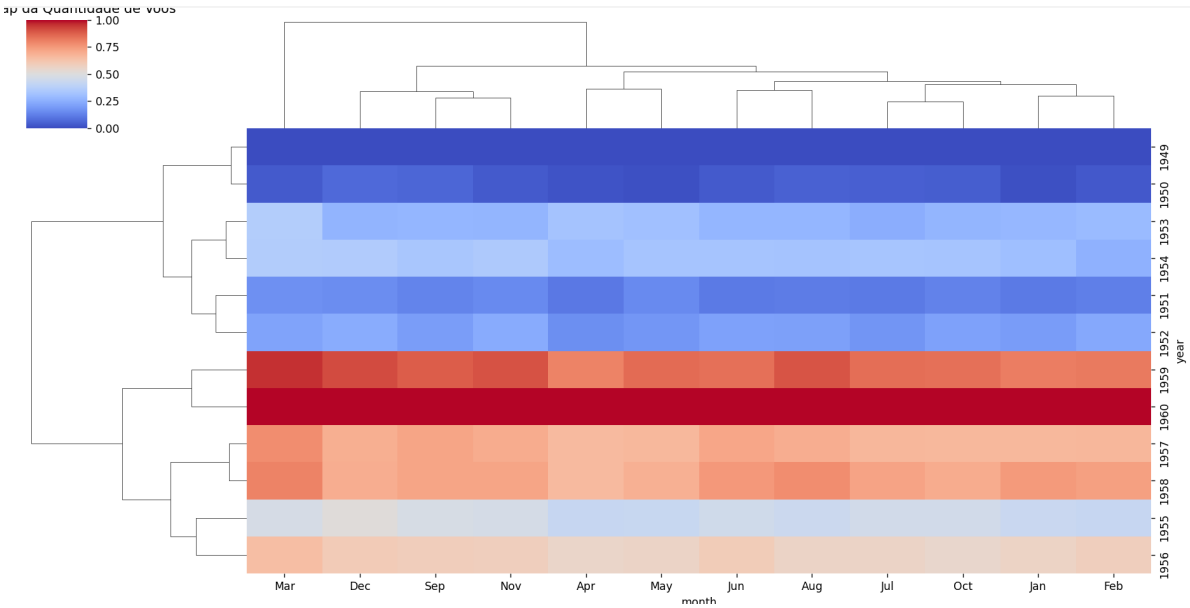

Figure 24: Ilustração do gráfico

é importante lembrar que para executar esse último código, é necessário instalar a biblioteca **SciPy** para executar a análise de clusters:

```
pip install scipy
```

O `clustermap` reagrupa automaticamente os dados, identificando padrões e correlações. Ele é útil para explorar conjuntos de dados grandes e desorganizados.

Conclusão

Os heatmaps são uma excelente ferramenta para visualizar dados categóricos em conjunto com uma variável contínua. O `sns.heatmap` permite uma visualização tridimensional intuitiva, enquanto o `sns.clustermap` ajuda a identificar padrões e agrupamentos nos dados.

Na próxima aula, abordaremos o tema de estilos no Seaborn.

06. Estilização

Bem-vindo à última aula do curso "Visualização de Dados Avançada com Seaborn". Nesta aula, abordaremos as opções de estilização disponíveis no Seaborn para melhorar a aparência dos nossos gráficos.

Função `set_style`

O Seaborn permite configurar diferentes estilos de apresentação para os gráficos por meio da função `sns.set_style()`. Abaixo estão alguns exemplos de estilos disponíveis:

Estilo `white`

```
import seaborn as sns
import matplotlib.pyplot as plt

dados = sns.load_dataset("tips")
sns.set_style("white")
sns.countplot(x="sex", data=dados)
plt.show()
```

Esse estilo remove as marcações dos ticks e deixa o fundo branco.

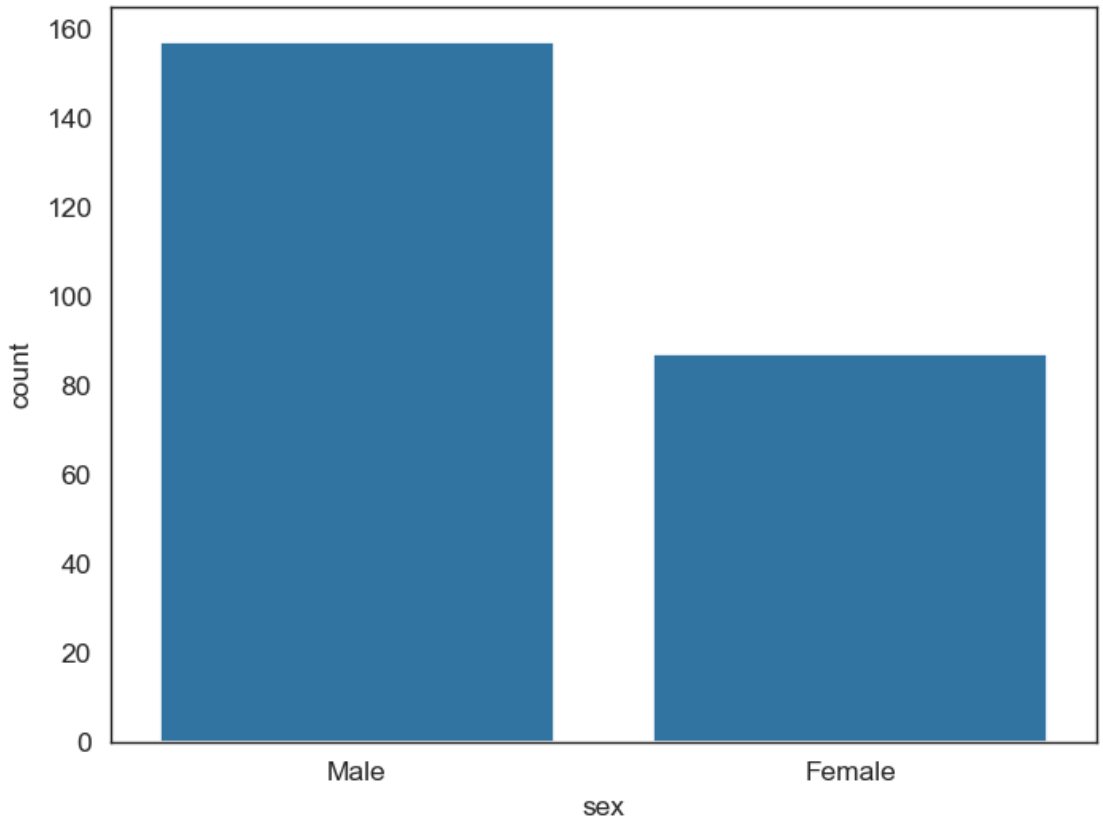

Figure 25: Ilustração do gráfico

Estilo `ticks`

```
sns.set_style("ticks")
sns.countplot(x="sex", data=dados)
plt.show()
```

Este estilo adiciona pequenas marcações nos eixos, melhorando a legibilidade.

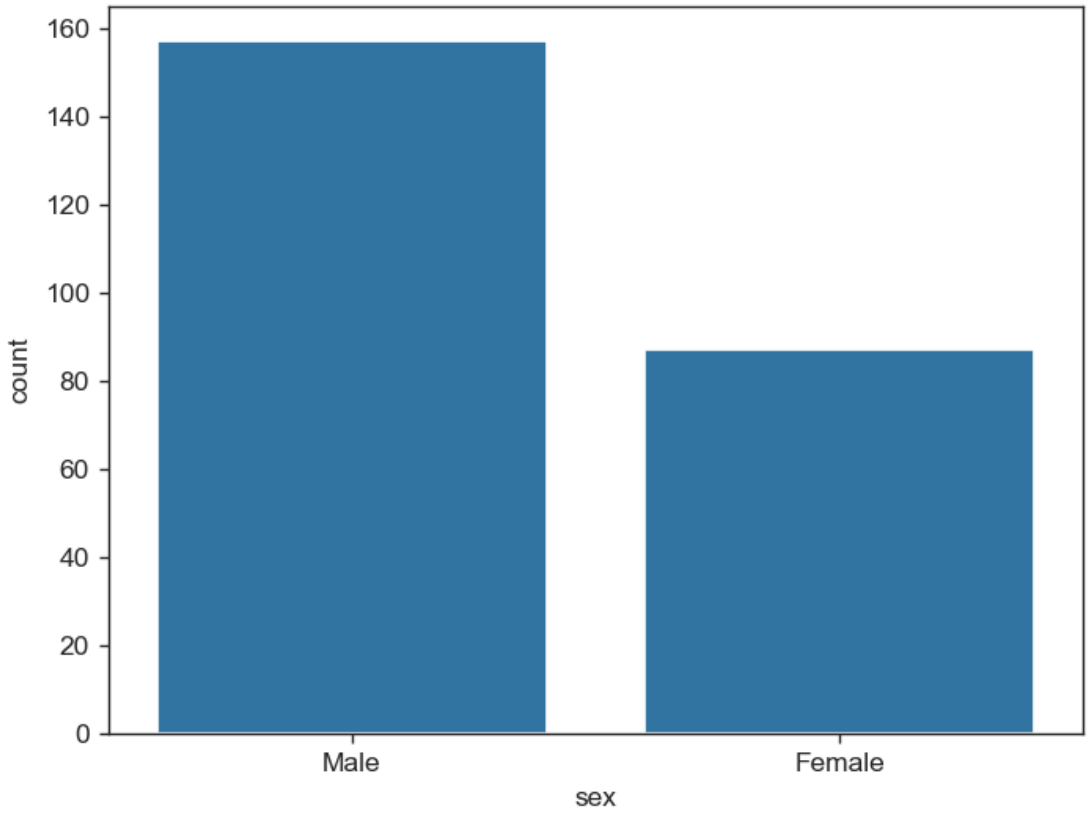

Figure 26: Ilustração do gráfico

Estilo `darkgrid`

```
sns.set_style("darkgrid")
sns.countplot(x="sex", data=dados)
plt.show()
```

O `darkgrid` adiciona um fundo levemente mais escuro com grades visíveis.

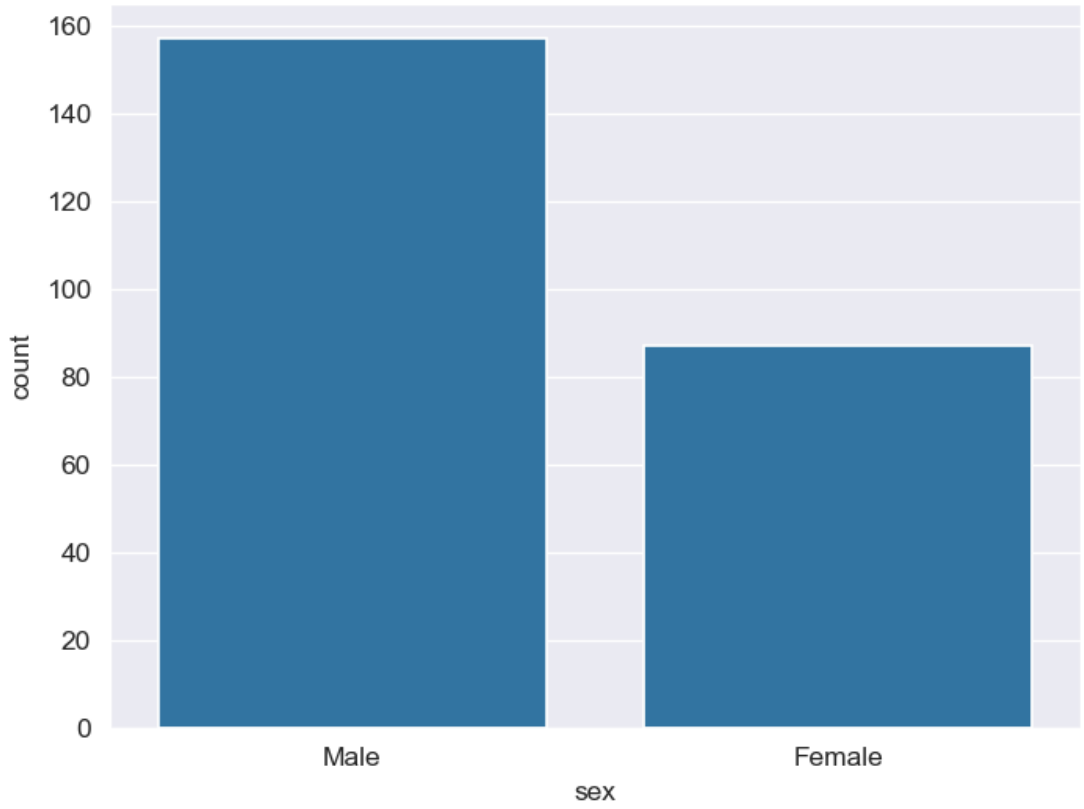

Figure 27: Ilustração do gráfico

Estilo `dark`

```
sns.set_style("dark")
sns.countplot(x="sex", data=dados)
plt.show()
```

Esse estilo remove a grade, deixando apenas um fundo escuro.

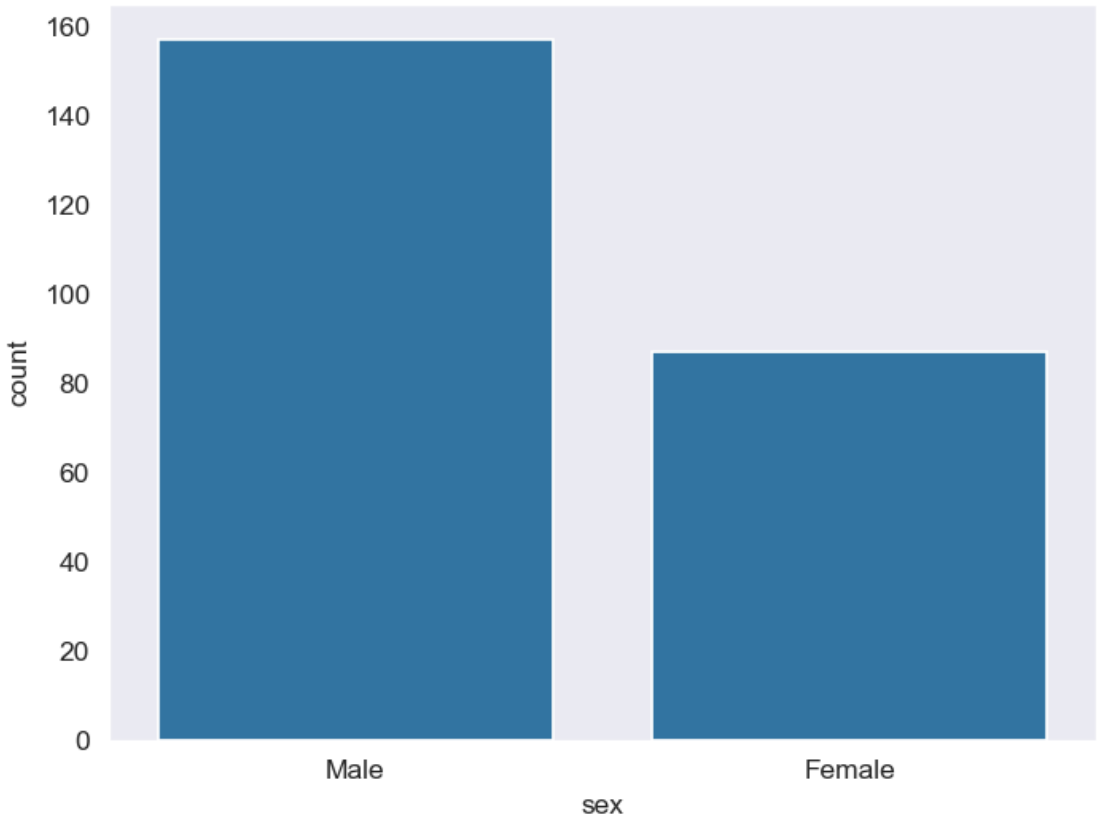

Figure 28: Ilustração do gráfico

Estilo `whitegrid`

```
sns.set_style("whitegrid")
sns.countplot(x="sex", data=dados)
plt.show()
```

Esse estilo adiciona uma grade clara sobre um fundo branco.

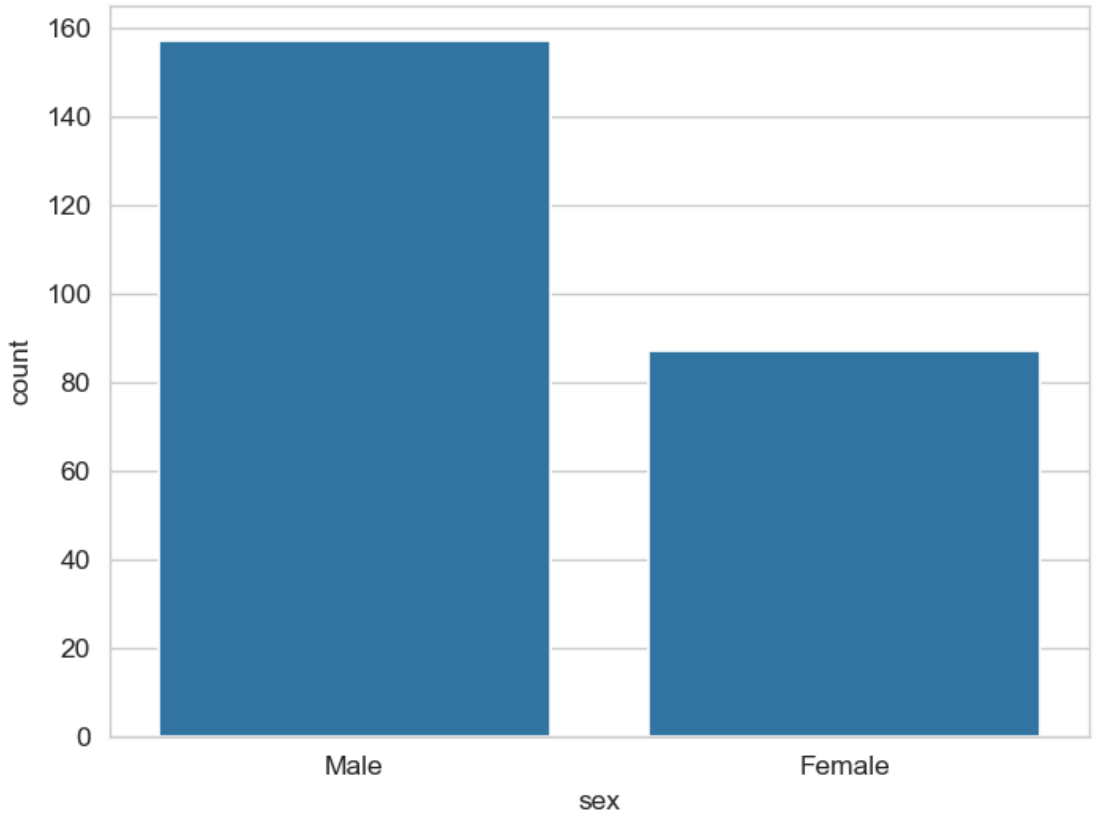

Figure 29: Ilustração do gráfico

Customização com Matplotlib

Para personalizações mais avançadas, podemos combinar o Seaborn com o Matplotlib:

```
fig, ax = plt.subplots(figsize=(8, 6))
sns.countplot(x="sex", data=dados, ax=ax)
ax.set_xlabel("Eixo X")
plt.show()
```

Dessa forma, conseguimos manipular diretamente o eixo do gráfico e ajustar tamanhos, rótulos e mais.

Configuração de Contexto com `set_context`

Outra opção é usar `sns.set_context()` para definir o contexto do gráfico:

```
sns.set_context("notebook", font_scale=1.5)
sns.countplot(x="sex", data=dados)
plt.show()
```

Isso permite alterar o tamanho de fontes e outros elementos visuais.

Considerações Finais

Cada estilo no Seaborn tem seu prós e contras, sendo ideal testar diferentes opções para encontrar a melhor para cada visualização. Além disso, a combinação com Matplotlib permite um controle ainda maior sobre a estilização.

Se você quiser explorar alternativas interativas para visualização de dados, uma boa opção é a biblioteca **Plotly**, que permite criar dashboards e animações interativas.

Esperamos que este curso tenha sido útil! Nos vemos em uma próxima jornada de aprendizado. Um grande abraço!